University Institute of Engineering

Department of Computer Science & Engineering

COMPUTER ORGANIZATION & ARCHITECTURE

(23CST-204/23ITT-204)

ER. SHIKHA ATWAL

E11186

ASSISTANT PROFESSOR

BE-CSE

INTERRUPTS

Before interrupts, the CPU had to wait for the signal to process by continuously checking for related hardware and software components (a method used earlier, referred to as "polling"). This method expends CPU cycles waiting, reducing our efficiency. The effective usage time of the CPU is reduced, reducing response time, and there's also increased power consumption.

The solution proposed to the above problem was the use of interrupts. In this method, instead of constantly checking for a signal, the CPU receives a signal from the hardware or software components. The process of sending the signal by hardware or software components is referred to as sending interrupts or interrupting the system.

INTERRUPTS

The interrupts divert the CPU's attention to the signal request from the running process. Once the interrupt signal is handled, the control is transferred back to the previous process to continue from the exact position where it had left off.

Types of Interrupts

The interrupts are of two types:

1. Hardware interrupt
2. Software interrupt

1. **Hardware Interrupt:** Interrupts generated by the hardware are referred to as hardware interrupts. The failure of hardware components and the completion of I/O can trigger hardware interrupts.

The two subtypes under it are:

- **Internal Interrupts:** These interrupts occur when there is an error due to some instruction. For example, overflows (register overflow), incorrect instructions code, etc. These types of interrupts are commonly referred to as traps.
- **External Interrupts:** These interrupts are the ones issued by hardware components. For example, when the I/O process is completed data transfer, an infinite loop in the given code, power failure, etc.

2. **Software Interrupt:** As the name suggests, these interrupts are caused by software, mostly in user mode. When a software interrupt occurs, the control is handed over to an interrupt handler (a part of the Operating System). Termination of programs or requests of certain services like the output to screen or using the printer can trigger it. These interrupts also have higher priority than hardware interrupts.

There are two types of software interrupts:

- **Normal Interrupts:** These interrupts are caused by software instructions and are made intentionally.
- **Exception Interrupts:** These interrupts are unplanned that occur during the execution of the program. An example of this is the divide by zero exception.

Handling of Interrupts by CPU

Let's assume there is a program composed of many instructions where we are processing the instructions one by one. Eventually, we have reached a certain instruction, and an interrupt occurs. Interrupts can occur for various reasons, for example, if you have a program that does the 'X' thing when the user presses the 'A' key on the keyboard. If we press the key 'A', the normal flow of the program has to be interrupted to process this new signal and do the 'X' thing. Later, we return to executing the instructions from where we left off before the previous process's interruption.

After the interrupt has occurred, the CPU needs to pass the control from the current process to service the interrupts instead of moving to the next instruction in the current process. Before transferring the control over to the interrupt generated program, we need to store the state of the currently running process.

The summary of the process followed during an interrupt is as below:

1. While executing some instructions of a process, an interrupt is issued.

2. Execution of the current instruction is completed, and the system responds to the interrupt.

3. The system sends an acknowledgement for the interrupt, and the interrupt signal from the source stops on receiving the acknowledgement.

4. The process's state of the current task is stored (register values, address of the next instruction to be processed when the control comes back to the process (Program counter in PC register), etc.), i.e., moved to the stack.

5. The processor now handles the interrupt and executes the interrupt generated program.

6. After handling the interrupt, the control is sent back to the point in the original process using the state information of the process that we saved earlier.

Interrupt Triggering Methods

Each interrupts signal input is designed to be triggered by either a logic signal level or a particular signal edge (level transition).

Level-sensitive inputs continuously request processor service so long as a particular (high or low) logic level is applied to the input.

Edge-sensitive inputs react to signal edges, a particular (rising or falling) edge will cause a service request to be latched. The processor resets the latch when the interrupt handler executes.

- **Level-triggered:** A level-triggered interrupt is requested by holding the interrupt signal at its particular (high or low) active logic level. A device invokes a level-triggered interrupt by driving the signal to and holding it at the active level. It negates the signal when the processor commands it, typically after the device has been serviced.

  The processor samples the interrupt input signal during each instruction cycle. The processor will recognize the interrupt request if the signal is asserted when sampling occurs.
  Level-triggered inputs allow multiple devices to share a common interrupt signal via wired-OR connections. The processor polls to determine which devices are requesting service. After servicing a device, the processor may again poll and, if necessary, service other devices before exiting the ISR.

- **Edge-triggered:** An edge-triggered interrupt is an interrupt signaled by a level transition on the interrupt line, either a falling edge (high to low) or a rising edge (low to high). A device wishing to signal an interrupt drives a pulse onto the line and releases it to its inactive state. If the pulse is too short to be detected by polled I/O, then special hardware may be required to detect it.

Advantages of Interrupts

Interrupts offer several advantages in system design:

- **Efficiency**: By using interrupts, the CPU can perform other tasks instead of constantly polling for events. This increases overall system efficiency and allows for better utilization of processing resources.
- **Responsiveness**: Interrupts enable the system to respond quickly to external events. This is particularly important in real-time applications where delays can lead to system failures or degraded performance.
- **Modularity**: Interrupts support modular design by allowing different parts of a system to handle specific events independently. This separation of concerns can simplify development and maintenance.
- **Power Saving**: In low-power systems, interrupts can help conserve energy by allowing the CPU to enter low-power modes when idle and wake up only when an interrupt occurs, reducing overall power consumption.

References

Reference Books:

- J.P. Hayes, “Computer Architecture and Organization”, Third Edition.
- Mano, M., “Computer System Architecture”, Third Edition, Prentice Hall.
- Stallings, W., “Computer Organization and Architecture”, Eighth Edition, Pearson Education.

Text Books:

- Carpinelli J.D,” Computer systems organization &Architecture”, Fourth Edition, Addison Wesley.
- Patterson and Hennessy, “Computer Architecture”, Fifth Edition Morgaon Kauffman.

Other References

- http://www.ecs.csun.edu/~cputnam/Comp546/Input-Output-Web.pdf
- http://www.ioenotes.edu.np/media/notes/computer-organization-and-architecture- coa/Chapter7-Input-Output-Organization.pdf
- https://www.geeksforgeeks.org/io-interface-interrupt-dma-mode/
- I/O Interface (Interrupt and DMA Mode) – GeeksforGeeks